

Guía de Despliegue en AWS

OE Inventory — Django 4.1 · MariaDB · PyMySQL

Hoja de ruta para publicar la aplicación en un servidor web en AWS, adaptada al estado actual del proyecto: estáticos externalizados, secretos en `.env`, subida de PDFs en `staff_docs` y esquema de BBDD gestionado manualmente (uso de `migrate --fake`).

1. Preparar la app para producción (en el código)

- **settings de producción:** `DEBUG = False`; `ALLOWED_HOSTS` con tu dominio; `SECRET_KEY` desde variable de entorno; `CSRF_TRUSTED_ORIGINS` con `https://tu-dominio`.
- **Seguridad HTTPS:** `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS` y `SECURE_PROXY_SSL_HEADER` (si hay balanceador delante). `X-Frame-Options` ya está bien.
- **Estáticos:** `collectstatic` a `STATIC_ROOT` y servirlos con `WhiteNoise` (lo más simple) o `S3` + `CloudFront`.
- **Media (importante):** los PDFs de `staff_docs` se guardan en disco local. En la nube no persiste ni escala bien → moverlos a `S3` con `django-storages`. Con una sola instancia el disco local sirve de momento.
- **requirements.txt:** añadir `gunicorn`, `whitenoise` (y `boto3/django-storages` si usas `S3`). Plantéate `mysqlclient` en vez de `PyMySQL` en producción.

2. Infraestructura AWS

- **RDS for MariaDB/MySQL:** tu base de datos gestionada.
- **Servidor de aplicación:** `gunicorn` (WSGI) detrás de `nginx`, gestionado con `systemd`.
- **HTTPS:** certificado en `ACM` + `Application Load Balancer`, o `certbot/Let's Encrypt` en `nginx`.
- **Secretos:** el `.env` vía `SSM Parameter Store` o `Secrets Manager` (no subir `.env` al repo).

AVISO CLAVE sobre la BBDD: tus modelos son `managed=True` pero la BBDD real tiene columnas gestionadas a mano (usáis `migrate --fake`). En una RDS nueva NO levantes el esquema con migraciones desde cero: haz un `DUMP` de la MariaDB actual y RESTAÚRALO en RDS para conservar datos y estructura. Verifica la versión de MariaDB compatible.

3. Tres caminos según cuánto quieras gestionar

Opción	Esfuerzo	Cuándo
EC2 única (<code>nginx</code> + <code>gunicorn</code> + <code>systemd</code>) + RDS + S3	Medio	App interna, tráfico bajo. Control total.
Elastic Beanstalk (plataforma Python) + RDS	Bajo	Que AWS gestione despliegue/escalado. Recomendado para empezar.
ECS Fargate (Docker) + ALB + RDS + S3	Alto	Contenedores, CI/CD, escalado serio.

4. Post-despliegue

- `collectstatic`, aplicar/verificar el esquema y crear superusuario si hace falta.
- Backups automáticos de RDS; grupos de seguridad (solo 443 público; la DB accesible solo desde la app); logs a `CloudWatch` (ya tienes `LOGGING` configurado).
- Dominio en `Route 53`.

Recomendación

Al ser una aplicación **interna de inventario** con tráfico moderado, lo más equilibrado es empezar por **Elastic Beanstalk + RDS (restaurando el dump) + S3 para los PDFs**. El primer paso que no depende de AWS — y que se puede dejar listo ya — es la preparación del código: un settings de producción con WhiteNoise/S3, el requirements.txt actualizado y pasar **manage.py check --deploy**.

Documento generado para OE Inventory.